# Red Pitaya STEMlab 125-14 PRO Gen 2 — Road Test Test Plan & Review Proposal

**Red Pitaya STEMlab 125-14 PRO Gen 2 Starter Kit + QSPI eMMC Module Gen 2**
*Review angle: Can one board replace a bench full of instruments?*

| | |
|---|---|
| **Product Under Test** | Red Pitaya STEMlab 125-14 PRO Gen 2 Starter Kit |
| **Review Duration** | 6 weeks from receipt of hardware |
| **Primary Skills** | Software Engineering, Electronics (Hobbyist) |
| **Reference Equipment Available** | TinyVNA, TinySA, multimeter, oscilloscope probes (supplied), Arduino/ESP32 dev boards |
| **Personal Motivation** | Inner city dweller with limited bench space, could a software-defined instrument reach pairity with a bench full of instruments? |

---

## 1. Review Overview

Cost of living means that budgets for large bench spaces and an accumulation of equipment are limited in today's world. Is there room where one piece of hardware can take the role of many instruments — and even become the prototyping platform or the project being built? Red Pitaya's STEMlab may be able to live up to this.

This review approaches the STEMlab 125-14 PRO Gen 2 from the perspective of a **software engineer with hobbyist electronics experience** — with limited experience and access to existing bench equipment — as a first and definitive option for a tool that does it all:

> *Does the Red Pitaya genuinely replace dedicated instruments, or does it approximate them with asterisks?*

To answer that honestly, every capability claim is tested against a real signal, cross-validated where possible against reference instruments (TinyVNA, TinySA), and measured for accuracy against known values. The QSPI eMMC module is treated as a first-class part of the review.

The review also explores one level below the web apps: loading an alternative FPGA bitstream from GitHub, and establishing a GPS-disciplined frequency reference — connecting hobbyist maker instincts to the platform's deeper capabilities.

---

## 2. Background & Motivation

Rather than accumulating test equipment, my hobbyist electronics projects have been mostly improvisational — and sometimes stalled when measurement was needed. In the element14 Smart Security and Surveillance challenge, debugging UART communications while rewriting a HAL (Hardware Abstraction Layer) for the MAX32630FTHR in Rust reached the point where I coded a rudimentary oscilloscope on an Arduino's ADC just to see the signal. Without real

instrumentation, a misbehaving I2C bus or a PWM edge arriving a microsecond late is simply invisible.

The STEMlab PRO Gen 2 claims to cover all of that:

- Oscilloscope
- Signal / Function Generator
- Spectrum Analyzer
- Bode Plotter
- Logic Analyzer
- LCR Meter, LCR (inductance (L), capacitance (C), and resistance (R))

That is potentially 6 instruments in one device, controlled via a browser, programmable via Python and Jupyter, and open at the FPGA level. This review is simply trying to find out if it genuinely can — and whether it does so without getting in the way.

---

## 3. Instruments Under Review & What Each Test Covers

### 3.1 Scope of "Instrument Replacement" Testing

| Instrument Role | Red Pitaya App | Test Method | Reference |
|---|---|---|---|
| Oscilloscope (2ch) | Built-in | Self-loopback at known frequencies, risetime measurement | Spec: 60 MHz BW (Bandwidth) |
| Signal Generator | Built-in | Feed into spectrum analyser and scope simultaneously | Cross-check frequency and amplitude |
| Spectrum Analyzer | Built-in | Known signal from OUT1, sweep to 60 MHz | TinySA cross-validation in overlap region |
| Bode Plotter | Built-in | RC filter sweep, measure -3dB point vs calculated | Hand-calculated expected value |
| Logic Analyzer | Built-in | I2C from ESP32/Arduino to sensor, decode | Compare decoded output vs known sensor data |
| LCR Meter | Built-in | Measure known resistors, capacitors, inductors from parts bin | Cross-check with datasheet tolerances |

### 3.2 Instruments the Red Pitaya Cannot Replace (Honest Limits)

- **Spectrum above 62 MHz**: The 125 MS/s ADC cannot sample above ~60 MHz (Nyquist). The TinySA covers to 960 MHz and beyond. The review will explicitly plot this limit with a side-by-side comparison.
- **VNA capability**: The STEMlab supports VNA via the Pavel Demin community app, but requires a separate SWR bridge (official module ~€100, or a DIY resistive bridge). Not tested in this review.
- **DC power supply**: No output regulation capability.
- **True RMS multimeter**: The 12-bit extension connector ADC (0–3.5V) is not a precision voltmeter replacement.

- **High-voltage measurements**: Input is ±1V standard / ±20V HV range — not a replacement for a fully isolated scope.

---

## 4. Test Procedure

### Week 1: Unboxing, Setup & First Impressions

#### Phase 1A: Hardware Assessment

1. Unbox, photograph all included items against the manifest
2. Fit QSPI eMMC module — document the physical installation process and M2 screw fit
3. Power up, connect via Ethernet, access web interface — note time-to-operational from cold start
4. Document first impressions: build quality, SMA connector feel, USB-C power experience, web UI responsiveness

#### Phase 1B: Storage Subsystem Comparison (eMMC vs MicroSD)

1. Boot from pre-installed MicroSD, note boot time
2. Flash OS image to eMMC module, switch boot, note boot time comparison
3. Run `dd` sequential read/write benchmark on both storage devices from SSH shell
4. Document: boot time difference, read/write throughput, practical implications

#### Phase 1C: Network & Remote Access

1. Test Ethernet and WiFi dongle connectivity
2. Measure web UI latency from local network
3. Document remote access options (SSH, web interface, SCPI over TCP)

---

### Week 2: Oscilloscope, Signal Generator & Spectrum Analyzer

#### Phase 2A: Self-Loopback Calibration Baseline

1. Connect OUT1 → IN1 using supplied SMA cable and T-adapters
2. Generate sine waves at 1 kHz, 10 kHz, 100 kHz, 1 MHz, 10 MHz, 50 MHz
3. At each frequency: measure amplitude (expected ±1V), frequency accuracy (expected within crystal tolerance), phase noise (qualitative)
4. Record: measured vs expected, note any deviation trends with frequency
5. This is the foundation — every subsequent measurement is calibrated against this baseline

#### Phase 2B: Bandwidth Boundary Test

1. Sweep OUT1 from 1 MHz to 65 MHz in 1 MHz steps
2. Read amplitude from IN1 — plot the rolloff curve
3. Identify the actual -3dB point vs the specified 60 MHz
4. Document: does the hardware meet its own spec?

#### Phase 2C: Spectrum Analyzer vs TinySA Cross-Validation

1. Generate a known signal (10 MHz sine) on OUT1

2. Capture via Red Pitaya spectrum analyzer — record peak amplitude, noise floor, spurious content
3. Connect same signal to TinySA
4. Compare: frequency accuracy, amplitude reading, noise floor dB figure
5. Document: where do they agree? Where do they diverge?

---

**Week 3: Bode Plotter, LCR Meter & Logic Analyzer**

**Phase 3A: Bode Plotter — RC Filter Verification**

1. Build a simple RC low-pass filter (e.g. 1kΩ + 100nF → calculated -3dB at ~1.59 kHz)
2. Connect: OUT1 → filter input, filter output → IN1
3. Run Bode sweep from 100 Hz to 100 kHz
4. Measure: -3dB frequency from Bode plot vs hand-calculated value
5. Repeat with a bandpass LC filter — compare measured vs simulated response

**Phase 3B: LCR Meter Accuracy**

1. Select 10 resistors, 5 capacitors, 3 inductors from parts bin — all with known datasheet values
2. Measure each with Red Pitaya LCR app
3. Cross-check: resistors with multimeter, capacitors and inductors with LCR readings vs datasheet tolerance bands
4. Tabulate: measured vs expected, note any systematic offset

**Phase 3C: Logic Analyzer — Protocol Decode**

1. Wire an I2C temperature sensor (e.g. DS18B20 or BMP280) to the extension connector digital IO pins
2. Run I2C read from an Arduino/ESP32
3. Capture with Red Pitaya logic analyzer, decode I2C frames
4. Verify: decoded register values match expected sensor output
5. Repeat with SPI device — document which protocols decode cleanly vs which require work

---

**Week 4: Python API, Jupyter & FPGA Layer**

**Phase 4A: Python/Jupyter Scripting**

1. Access built-in Jupyter environment from browser
2. Write a script: sweep OUT1 frequency, read peak amplitude from IN1 ADC at each step — a software-implemented Bode plotter
3. Plot results with matplotlib, compare to the built-in Bode app output
4. Document: what can you do in Python that the built-in apps can't?

**Phase 4B: GPS-Disciplined Frequency Reference (Maker Metrology)**

1. Connect a cheap u-blox GPS module (NEO-6M or NEO-M8N) to the extension connector — GPS 1PPS output to a digital IO pin
2. Write a Python script to timestamp the 1PPS pulse against the system clock

3. Measure the Red Pitaya's own 125 MHz crystal accuracy vs GPS-derived reference
4. This contextualises all frequency measurements in the review with a traceable reference — and connects to the broader question of what "accurate" means at the maker level
5. Document: crystal accuracy in ppm, how much this matters for each instrument function

**Phase 4C: Loading an Alternative FPGA Bitstream**

1. Clone the Red Pitaya FPGA repository from GitHub
2. Load a pre-built community bitstream via SSH (fpgautil command) — the LED blink example
   - following https://redpitaya-knowledge-base.readthedocs.io/en/latest/learn_fpga/4_lessons/LedBlink.html
   - using https://pavel-demin.github.io/red-pitaya-notes/led-blinker/
   - or the more approachable https://antonpotocnik.com/?p=487360
3. Document the full procedure: what's required, how long it takes, what breaks and what doesn't
4. Reload factory image, verify all apps restore cleanly
5. Discuss: what this unlocks — the TDC project, custom DSP, community SDR apps

---

**Week 5: Audio & Signal Generation**

**Phase 5A: Shepard Tone — AWG as Synthesiser**

1. Generate a Shepard tone in Python — sine waves summed an octave apart with overlapping amplitude envelopes, output via OUT1
2. Capture on the spectrum analyzer — the fading partials make the auditory illusion visible: each harmonic fades out just as the next appears an octave up
3. Document: a signal the built-in function generator cannot produce, generated in a Jupyter notebook in minutes

**Phase 5B: Chirp & DTMF — Practical Signal Tests**

1. Generate a chirp (sine swept 20 Hz → 20 kHz) and visualise on the scope and spectrogram — the same technique used internally by the Bode plotter
2. Generate DTMF tones (two-frequency phone dial tones, e.g. digit "5" = 770 Hz
   - 1336 Hz) and identify both components on the spectrum analyzer
3. Document: what additive synthesis via the AWG reveals about the spectrum analyzer's resolution and dynamic range at audio frequencies

---

**Week 6: Final Benchmarks**

**Phase 6A: Comprehensive Accuracy Summary**

| Measurement | Expected | Measured | Error | Verdict |
|---|---|---|---|---|
| Frequency accuracy (10 MHz) | 10.000000 MHz | TBD | TBD | TBD |
| Amplitude accuracy (±1V) | 1.000 V pk | TBD | TBD | TBD |
| Bode -3dB (RC filter) | 1.592 kHz | TBD | TBD | TBD |

| Measurement | Expected | Measured | Error | Verdict |
|---|---|---|---|---|
| LCR — 10kΩ resistor | 10.00 kΩ | TBD | TBD | TBD |
| Spectrum peak vs TinySA | Reference | TBD | TBD | TBD |
| eMMC boot time | Faster than SD | TBD | TBD | TBD |
| Crystal accuracy (pre-GPS) | ±50 ppm | TBD | TBD | TBD |

**Phase 6B: Review Documentation**

The final element14 community review will include:

- Unboxing and hardware quality assessment
- Full instrument-by-instrument test results with measured data, graphs, and photographs
- TinySA cross-validation results — honest side-by-side comparison
- GPS-disciplined frequency reference setup — a maker metrology explainer
- FPGA bitstream loading walkthrough — SSH procedure documented step by step
- Python/Jupyter scripting examples with working code (GitHub repository)
- Shepard tone, chirp, and DTMF signal generation demos with spectrum analysis
- Honest "instrument replacement scorecard" — what it replaces well, what it approximates, what it cannot do
- Bill of materials for any additional items used in testing

---

## 5. Risk Register

| Risk | Likeli | Mitigation |
|---|---|---|
| GPS 1PPS integration requires kernel driver work | Low | u-blox modules output clean 3.3V logic 1PPS; readable as a simple GPIO interrupt with no special driver |
| FPGA bitstream load bricks the board | Low | Factory image restoration is a documented single command; eMMC module provides additional boot fallback |
| Crystal accuracy within spec — GPS test uninteresting | Low | The test has value regardless of outcome: it establishes traceability context and demonstrates the extension connector's capability |

---

## 6. Qualifications & Relevant Experience

- **Software Engineering (Professional):** Engineering leader with experience in system architecture, API design, and technical documentation. Produces written reviews that are structured, reproducible, and honestly quantified.
- **Electronics & Maker Projects (Hobbyist):** Active participant in element14 design challenges. Recent projects include runner up in Smart Security and Surveillance challenge using MAX32630FTHR and currently taking part in CAN-bus smart greenhouse controller as well as doing my final write up for the E14 Al Foil aTtenuator DevKit

- **Reference Instruments Available:** TinyVNA and TinySA available for cross-validation, I happen to have them on hand.

This review is not a feature walkthrough. It is an accuracy and usability audit with quantified results, written for engineers who want to know whether this board earns a place on their bench.

---

## 7. Deliverables & Timeline

| Week | Deliverable |
|---|---|
| Week 1 | Unboxing post + hardware quality notes + eMMC vs MicroSD storage benchmark + first impressions of web interface |
| Week 2 | Oscilloscope and signal generator accuracy results + spectrum analyzer vs TinySA cross-validation + bandwidth rolloff curve |
| Week 3 | Bode plotter RC filter verification + LCR meter accuracy table + logic analyzer I2C/SPI decode walkthrough |
| Week 4 | Python/Jupyter scripting examples + GPS frequency reference setup + FPGA bitstream loading walkthrough |
| Week 5 | Shepard tone, chirp & DTMF signal generation — AWG + spectrum analyzer demos with Jupyter notebooks |
| Week 6 | Final review: complete accuracy scorecard, instrument replacement verdict, GitHub code repository, all measurements consolidated |

All posts published to the element14 community road test group with full measurement data, photographs, and code. GitHub repository made public on completion.

---

*Current ideation and working notes: github.com/saramic/learning/tree/master/competitions*